Database schema

PostgreSQL, 19 application tables plus Alembic's `alembic_version` (current head `c9e3a1b5d7f4`). All primary keys are UUIDs generated by the application. The entity-relationship diagram is in erd.md; tenant isolation is described in multi-tenancy.md.

Conventions

- **Multi-tenant.** Every tenant-owned table has `company_id` (FK → `companies.id`). Row-Level Security is enabled on them with a `tenant_isolation` policy for `fddt_app` (`company_id = fddt_current_company_id()` , the transaction-local `app.current_company_id`) and a `platform_all` policy for `fddt_platform` . No role has BYPASSRLS.
- **Insert-only tables** (the app roles are granted only SELECT + INSERT): `audit_log` , `case_risk_assessments` , `case_reports` , `risk_rules` (rule edits insert a new version).
- **JSON columns** (`jsonb`) hold check output and snapshots; their shapes are documented in the pipeline docs.
- **Bounding boxes** anywhere in JSON use one convention: `{page (1-based), x, y, width, height}` as 0-1 fractions of the page, origin top-left.
- **Enums** are native Postgres enums; their values are listed under each table and again in the index at the bottom.

Table	Tenancy	`fddt_app`	Purpose
`companies`	RLS (own row)	SELECT	One row per client company (tenant). `is_active = false` suspends it: its users can't sign in and existing tokens stop working.
`company_usage_stats`	tenant · RLS	SELECT, INSERT, UPDATE	Per-company, per-UTC-day usage counters (cases, documents, files, storage bytes) for the platform billing dashboard; bumped in the same transaction as each upload/case/report and reconciled nightly.
`users`	tenant · RLS	SELECT	Accounts (email + bcrypt-hashed password). Company users have a `company_id` and a ranked role (`user` / `reviewer_l1` / `reviewer_l2`); platform admins have `role = platform_admin` and no company (CHECK constraint).
`cases`	tenant · RLS	SELECT, INSERT, UPDATE	The unit of work: one submission of one or more documents. Carries workflow `status` , the owning reviewer tier `assigned_tier` and the latest `risk_tier` .
`documents`	tenant · RLS	SELECT, INSERT, UPDATE	One row per uploaded PDF. The original bytes live in Blob Storage (immutable); this row holds the SHA-256, the extraction results (`extracted_fields` , `ocr_text`) and `processing_status` .
`document_checks`	tenant · RLS	SELECT, INSERT, UPDATE	One row per (document, check type) — the latest run of each automated check (extraction stages, every forensic check, signature detection…), upserted so a redelivered task never duplicates it. `result` is the check's raw JSON output; the run history is in `audit_log` .
`document_page_hashes`	tenant · RLS	SELECT, INSERT, UPDATE	Perceptual hash (pHash) of every rendered PDF page, one per (document, page), used to find duplicate/near-duplicate pages within the same company.
`cross_document_findings`	tenant · RLS	SELECT, INSERT, UPDATE, DELETE	Case-level reconciliation results: a shared field (amount, date, issuer) that disagrees between documents in the same case.

Table	Tenancy	`fddt_app`	Purpose
`issuer_registry`	tenant · RLS	SELECT, INSERT, UPDATE	A company's known vendors/schools/etc. that extracted issuer names are fuzzy-matched against. Rows are deactivated, never deleted. A new company starts empty.
`risk_rules`	tenant · RLS	SELECT, INSERT	A company's configurable scoring rules. Rows are immutable and versioned: an edit inserts `version + 1`; the current rule is the highest version. Seeded from `risk_rule_templates` when the company is created.
`risk_rule_templates`	platform-only	—	PLATFORM-LEVEL (no `company_id`): the default rule set copied into every NEW company's `risk_rules`. Edited by platform admins; edits never change existing companies. Only `fddt_platform` can access it.
`risk_settings`	tenant · RLS	SELECT, INSERT, UPDATE	One row per company holding its `medium` / `high` score thresholds.
`case_risk_assessments`	tenant · RLS	SELECT, INSERT	One row per scoring run of a case (insert-only history; unique per (case, evidence fingerprint)): score, tier, frozen reason text and a snapshot of the exact rule versions and thresholds used.
`risk_scores`	tenant · RLS	SELECT	LEGACY. Created by the initial migration before the versioned engine existed; nothing in the application reads or writes it. `case_risk_assessments` replaced it.
`case_actions`	tenant · RLS	SELECT, INSERT	Reviewer-facing history of human actions on a case (approve / reject / escalate …) with the actor's role at the time. Complements, and is also mirrored into, `audit_log`.
`case_reports`	tenant · RLS	SELECT, INSERT	History of generated per-case forensic PDF reports (insert-only). The PDF lives in Blob Storage under `companies/{company}/cases/{case}/reports/`; this row stores its SHA-256, size and the assessment it reflects.
`audit_log`	tenant · RLS	SELECT, INSERT	Append-only system of record: every automated check result, human action and settings change is inserted here and never updated or deleted. `company_id` NULL = a platform-level event (company created, template edits, usage reconciliation, platform-admin access), visible only to platform admins; a company user's *user created/updated* goes into that company's log.
`signature_references`	tenant · RLS	SELECT, INSERT, UPDATE	A signature/stamp region drawn on a document, cropped and stored as a PNG, used as the reference for comparisons.
`signature_matches`	tenant · RLS	SELECT, INSERT, UPDATE	Result of comparing one reference against the signature region of another document in the same case (vision-model verdict or pixel-identical reuse verdict); one row per (reference, document, scope).

companies

One row per client company (tenant). `is_active = false` suspends it: its users can't sign in and existing tokens stop working.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK

Column	Type	Null	Default	Notes
`name`	varchar(255)	no		unique
`is_active`	boolean	no	`true`	
`created_at`	timestamptz	no	`now()`	
`updated_at`	timestamptz	no	`now()`	

- **Index** `companies_name_key` on (name) — unique
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

company_usage_stats

Per-company, per-UTC-day usage counters (cases, documents, files, storage bytes) for the platform billing dashboard; bumped in the same transaction as each upload/case/report and reconciled nightly.

Column	Type	Null	Default	Notes
`company_id`	uuid	no		PK, FK → `companies.id`
`usage_date`	date	no		PK
`cases_created`	integer	no	`0`	
`documents_uploaded`	integer	no	`0`	
`files_stored`	integer	no	`0`	
`storage_bytes`	bigint	no	`'0'::bigint`	
`updated_at`	timestamptz	no	`now()`	

- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT, UPDATE · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

users

Accounts (email + bcrypt-hashed password). Company users have a `company_id` and a ranked role (`user` / `reviewer_l1` / `reviewer_l2`); platform admins have `role = platform_admin` and no company (CHECK constraint).

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`email`	varchar(320)	no		unique
`hashed_password`	varchar(255)	no		
`full_name`	varchar(255)	yes		
`role`	enum `user_role`	no		

Column	Type	Null	Default	Notes
`is_active`	boolean	no		
`company_id`	uuid	yes		FK → `companies.id`

- **Index** `ix_users_company_id` on (company_id)
- **Index** `ix_users_email` on (email)
- **Index** `users_email_key` on (email) — unique
- **Check** `ck_users_platform_admin_has_no_company` : `CHECK (((role = 'platform_admin'::user_role) = (company_id IS NULL)))`
- **Enum** `user_role` : `user` , `reviewer_l1` , `reviewer_l2` , `admin` , `platform_admin`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

cases

The unit of work: one submission of one or more documents. Carries workflow `status` , the owning reviewer tier `assigned_tier` and the latest `risk_tier` .

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`case_number`	varchar(64)	no		unique
`submitted_by_user_id`	uuid	no		FK → `users.id`
`status`	enum `case_status`	no		
`risk_tier`	enum `risk_tier`	yes		
`external_ref_id`	varchar(255)	yes		
`case_type`	enum `case_type`	no		
`assigned_tier`	enum `case_tier`	no	`'l1'::case_tier`	
`company_id`	uuid	no		FK → `companies.id`

- **Index** `cases_case_number_key` on (case_number) — unique
- **Index** `ix_cases_case_number` on (case_number)
- **Index** `ix_cases_company_id` on (company_id)
- **Enum** `case_status` : `submitted` , `under_automated_review` , `pending_manual_review` , `auto_approved` , `escalated` , `under_investigation` , `approved` , `rejected` , `closed`
- **Enum** `risk_tier` : `low` , `medium` , `high`
- **Enum** `case_type` : `school_document` , `vendor_invoice` , `commercial_invoice` , `procurement_documentation` , `quotation` , `travel_reimbursement` , `other`
- **Enum** `case_tier` : `l1` , `l2`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)

- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

documents

One row per uploaded PDF. The original bytes live in Blob Storage (immutable); this row holds the SHA-256, the extraction results (`extracted_fields` , `ocr_text`) and `processing_status` .

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`case_id`	uuid	no		FK → `cases.id`
`uploaded_by_user_id`	uuid	no		FK → `users.id`
`original_filename`	varchar(512)	no		
`blob_storage_path`	varchar(1024)	no		
`file_hash`	varchar(64)	no		
`content_type`	varchar(255)	yes		
`file_size_bytes`	integer	yes		
`document_type`	varchar(128)	yes		
`processing_status`	enum `document_processing_status`	no		
`extracted_fields`	jsonb	yes		
`ocr_text`	text	yes		
`processing_error`	text	yes		
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_documents_case_id` on (case_id)
- **Index** `ix_documents_company_id` on (company_id)
- **Index** `ix_documents_file_hash` on (file_hash)
- **Enum** `document_processing_status`: `pending` , `processing` , `complete` , `failed`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

document_checks

One row per (document, check type) — the latest run of each automated check (extraction stages, every forensic check, signature detection…), upserted so a redelivered task never duplicates it. `result` is the check's raw JSON output; the run history is in `audit_log` .

Column	Type	Null	Default	Notes
`id`	uuid	no		PK

Column	Type	Null	Default	Notes
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`document_id`	uuid	no		FK → `documents.id`
`check_type`	enum `document_check_type`	no		
`status`	enum `document_check_status`	no		
`result`	jsonb	yes		
`confidence`	double precision	yes		
`error_message`	varchar(2048)	yes		
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_document_checks_company_id` on (company_id)
- **Index** `ix_document_checks_document_id` on (document_id)
- **Index** `uq_document_checks_document_check_type` on (document_id, check_type) — unique
- **Enum** `document_check_type`: `ocr_layout`, `classification`, `field_extraction`, `issuer_verification`, `signature_stamp_detection`, `metadata_forensics`, `error_level_analysis`, `copy_move_detection`, `duplicate_detection`, `ai_content_detection`, `field_validation`, `visual_inconsistency_review`
- **Enum** `document_check_status`: `pending`, `running`, `completed`, `failed`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

document_page_hashes

Perceptual hash (pHash) of every rendered PDF page, one per (document, page), used to find duplicate/near-duplicate pages within the same company.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`document_id`	uuid	no		FK → `documents.id`
`page_number`	integer	no		
`phash`	varchar(32)	no		
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_document_page_hashes_company_id` on (company_id)
- **Index** `ix_document_page_hashes_document_id` on (document_id)
- **Index** `ix_document_page_hashes_phash` on (phash)
- **Index** `uq_document_page_hashes_document_page` on (document_id, page_number) — unique
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)

- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

cross_document_findings

Case-level reconciliation results: a shared field (amount, date, issuer) that disagrees between documents in the same case.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`case_id`	uuid	no		FK → `cases.id`
`field_name`	varchar(128)	no		
`finding_type`	varchar(128)	no		
`severity`	enum `finding_severity`	no		
`description`	varchar(2048)	no		
`document_ids`	jsonb	yes		
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_cross_document_findings_case_id` on (case_id)
- **Index** `ix_cross_document_findings_company_id` on (company_id)
- **Enum** `finding_severity`: `info`, `low`, `medium`, `high`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT, UPDATE, DELETE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

issuer_registry

A company's known vendors/schools/etc. that extracted issuer names are fuzzy-matched against. Rows are deactivated, never deleted. A new company starts empty.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`name`	varchar(255)	no		
`tax_id`	varchar(64)	yes		
`type`	enum `issuer_registry_type`	no		
`name_arabic`	varchar(255)	yes		
`is_active`	boolean	no	`true`	
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_issuer_registry_company_id` on (company_id)
- **Index** `ix_issuer_registry_name` on (name)
- **Index** `ix_issuer_registry_name_arabic` on (name_arabic)
- **Enum** `issuer_registry_type` : `vendor` , `school` , `government` , `other`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT, UPDATE · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

risk_rules

A company's configurable scoring rules. Rows are immutable and versioned: an edit inserts `version + 1` ; the current rule is the highest version. Seeded from `risk_rule_templates` when the company is created.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`rule_id`	varchar(64)	no		
`category`	varchar(128)	no		
`condition`	jsonb	no		
`weight`	double precision	no		
`severity`	varchar(32)	no		
`reason_template`	varchar(1024)	no		
`is_active`	boolean	no		
`check_type`	varchar(64)	no		
`version`	integer	no		
`effective_from`	timestamptz	no		
`updated_by`	uuid	yes		FK → `users.id`
`change_note`	varchar(1024)	yes		
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_risk_rules_company_id` on (company_id)
- **Index** `ix_risk_rules_rule_id` on (rule_id)
- **Index** `uq_risk_rules_company_rule_id_version` on (company_id, rule_id, version) — unique
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT · `fddt_platform` : SELECT, INSERT

risk_rule_templates

PLATFORM-LEVEL (no `company_id`): the default rule set copied into every NEW company's `risk_rules` . Edited by platform admins; edits never change existing companies. Only `fddt_platform` can access it.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`rule_id`	varchar(64)	no		unique
`category`	varchar(128)	no		
`check_type`	varchar(64)	no		
`condition`	jsonb	no		
`weight`	double precision	no		
`severity`	varchar(32)	no		
`reason_template`	varchar(1024)	no		
`is_active`	boolean	no	`true`	
`updated_by`	uuid	yes		FK → `users.id`
`created_at`	timestamptz	no	`now()`	
`updated_at`	timestamptz	no	`now()`	

- **Index** `risk_rule_templates_rule_id_key` on (rule_id) — unique
- **Grants** `fddt_app` : — · `fddt_platform` : SELECT, INSERT, UPDATE

risk_settings

One row per company holding its `medium` / `high` score thresholds.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`medium_threshold`	integer	no		
`high_threshold`	integer	no		
`updated_by`	uuid	yes		FK → `users.id`
`company_id`	uuid	no		FK → `companies.id`, unique

- **Index** `ix_risk_settings_company_id` on (company_id)
- **Index** `uq_risk_settings_company_id` on (company_id) — unique
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT, UPDATE · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

case_risk_assessments

One row per scoring run of a case (insert-only history; unique per (case, evidence fingerprint)): score, tier, frozen reason text and a snapshot of the exact rule versions and thresholds used.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`case_id`	uuid	no		FK → `cases.id`
`raw_score`	double precision	no		
`score`	integer	no		
`tier`	enum `risk_tier`	no		
`triggered_reasons`	jsonb	no		
`risk_rules_version_snapshot`	jsonb	no		
`evidence_fingerprint`	varchar(64)	no		
`computed_at`	timestamptz	no		
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_case_risk_assessments_case_id` on (case_id)
- **Index** `ix_case_risk_assessments_company_id` on (company_id)
- **Index** `ix_case_risk_assessments_computed_at` on (computed_at)
- **Index** `ix_case_risk_assessments_evidence_fingerprint` on (evidence_fingerprint)
- **Index** `uq_case_risk_assessments_case_evidence` on (case_id, evidence_fingerprint) — unique
- **Enum** `risk_tier` : `low` , `medium` , `high`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT · `fddt_platform` : SELECT, INSERT

risk_scores

LEGACY. Created by the initial migration before the versioned engine existed; nothing in the application reads or writes it. `case_risk_assessments` replaced it.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`case_id`	uuid	no		FK → `cases.id`
`score`	integer	no		
`tier`	enum `risk_score_tier`	no		
`triggered_reasons`	jsonb	no		
`rules_snapshot`	jsonb	no		
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_risk_scores_case_id` on (case_id)
- **Index** `ix_risk_scores_company_id` on (company_id)

- **Enum** `risk_score_tier`: `low`, `medium`, `high`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

case_actions

Reviewer-facing history of human actions on a case (approve / reject / escalate …) with the actor's role at the time. Complements, and is also mirrored into, `audit_log`.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`case_id`	uuid	no		FK → `cases.id`
`actor_user_id`	uuid	yes		FK → `users.id`
`action_type`	enum `case_action_type`	no		
`notes`	varchar(4096)	yes		
`actor_role`	varchar(32)	yes		
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_case_actions_case_id` on (case_id)
- **Index** `ix_case_actions_company_id` on (company_id)
- **Enum** `case_action_type`: `assign`, `comment`, `approve`, `reject`, `escalate`, `reopen`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

case_reports

History of generated per-case forensic PDF reports (insert-only). The PDF lives in Blob Storage under `companies/{company}/cases/{case}/reports/`; this row stores its SHA-256, size and the assessment it reflects.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`case_id`	uuid	no		FK → `cases.id`
`generated_by_user_id`	uuid	no		FK → `users.id`
`generated_at`	timestamptz	no		
`blob_url`	text	no		
`file_size_bytes`	integer	no		
`page_count`	integer	no		
`report_sha256`	varchar(64)	no		

Column	Type	Null	Default	Notes
`risk_assessment_id`	uuid	yes		FK → `case_risk_assessments.id`
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_case_reports_case_id` on (case_id)
- **Index** `ix_case_reports_company_id` on (company_id)
- **Index** `ix_case_reports_generated_at` on (generated_at)
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT · `fddt_platform` : SELECT, INSERT

audit_log

Append-only system of record: every automated check result, human action and settings change is inserted here and never updated or deleted. `company_id` NULL = a platform-level event (company created, template edits, usage reconciliation, platform-admin access), visible only to platform admins; a company user's *user created/updated* goes into that company's log.

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`case_id`	uuid	yes		FK → `cases.id`
`document_id`	uuid	yes		FK → `documents.id`
`actor_user_id`	uuid	yes		FK → `users.id`
`event_type`	varchar(128)	no		
`event_data`	jsonb	yes		
`company_id`	uuid	yes		FK → `companies.id`

- **Index** `ix_audit_log_case_id` on (case_id)
- **Index** `ix_audit_log_company_id` on (company_id)
- **Index** `ix_audit_log_created_at` on (created_at)
- **Index** `ix_audit_log_document_id` on (document_id)
- **Index** `ix_audit_log_event_type` on (event_type)
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT · `fddt_platform` : SELECT, INSERT

signature_references

A signature/stamp region drawn on a document, cropped and stored as a PNG, used as the reference for comparisons.

Column	Type	Null	Default	Notes
`person_name`	text	no		
`signature_image_url`	text	yes		

Column	Type	Null	Default	Notes
`bounding_box`	jsonb	yes		
`source_document_id`	uuid	no		FK → `documents.id`
`source_case_id`	uuid	no		FK → `cases.id`
`created_by`	uuid	no		FK → `users.id`
`is_library`	boolean	no	`false`	
`id`	uuid	no		PK
`created_at`	timestamptz	no		
`updated_at`	timestamptz	no		
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_signature_references_company_id` on (company_id)
- **Index** `ix_signature_references_source_case_id` on (source_case_id)
- **Index** `ix_signature_references_source_document_id` on (source_document_id)
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

signature_matches

Result of comparing one reference against the signature region of another document in the same case (vision-model verdict or pixel-identical reuse verdict); one row per (reference, document, scope).

Column	Type	Null	Default	Notes
`id`	uuid	no		PK
`document_id`	uuid	no		FK → `documents.id`
`case_id`	uuid	no		FK → `cases.id`
`signature_reference_id`	uuid	no		FK → `signature_references.id`
`comparison_scope`	enum `comparison_scope`	no		
`result`	enum `signature_match_result`	no		
`reasoning`	text	yes		
`compared_at`	timestamptz	no		
`company_id`	uuid	no		FK → `companies.id`

- **Index** `ix_signature_matches_case_id` on (case_id)
- **Index** `ix_signature_matches_company_id` on (company_id)
- **Index** `ix_signature_matches_document_id` on (document_id)
- **Index** `ix_signature_matches_signature_reference_id` on (signature_reference_id)
- **Index** `uq_signature_matches_ref_doc_scope` on (signature_reference_id, document_id, comparison_scope) — unique

- **Enum** `comparison_scope`: `in_case`, `library`
- **Enum** `signature_match_result`: `consistent`, `possibly_consistent`, `inconsistent`, `cannot_determine`, `identical_reuse`, `reused_different_signer`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

Enum index

Enum	Values
`case_action_type`	`assign`, `comment`, `approve`, `reject`, `escalate`, `reopen`
`case_status`	`submitted`, `under_automated_review`, `pending_manual_review`, `auto_approved`, `escalated`, `under_investigation`, `approved`, `rejected`, `closed`
`case_tier`	`l1`, `l2`
`case_type`	`school_document`, `vendor_invoice`, `commercial_invoice`, `procurement_documentation`, `quotation`, `travel_reimbursement`, `other`
`comparison_scope`	`in_case`, `library`
`document_check_status`	`pending`, `running`, `completed`, `failed`
`document_check_type`	`ocr_layout`, `classification`, `field_extraction`, `issuer_verification`, `signature_stamp_detection`, `metadata_forensics`, `error_level_analysis`, `copy_move_detection`, `duplicate_detection`, `ai_content_detection`, `field_validation`, `visual_inconsistency_review`
`document_processing_status`	`pending`, `processing`, `complete`, `failed`
`finding_severity`	`info`, `low`, `medium`, `high`
`issuer_registry_type`	`vendor`, `school`, `government`, `other`
`risk_score_tier`	`low`, `medium`, `high`
`risk_tier`	`low`, `medium`, `high`
`signature_match_result`	`consistent`, `possibly_consistent`, `inconsistent`, `cannot_determine`, `identical_reuse`, `reused_different_signer`
`user_role`	`user`, `reviewer_l1`, `reviewer_l2`, `admin`, `platform_admin`

Schema notes

- `user_role` **still contains** `admin`**.** The single-tenant `admin` role was migrated to `platform_admin`; Postgres cannot drop an enum value, so `admin` remains in the type but no row uses it and the application never assigns it.

- `cases.case_number` **and** `users.email` carry a `UNIQUE` constraint *plus* a separate non-unique index from the original migrations; uniqueness is enforced either way.
- `risk_scores` exists but nothing in the application uses it. It is safe to leave; dropping it needs a migration.
- **No** `notifications` **table.** Notifications are not part of the current build.
- **Enum values added by later migrations** use `ALTER TYPE … ADD VALUE`; a downgrade leaves them in place.
- **New tenant-owned tables** need `TenantScopedMixin`, RLS with a `tenant_isolation` and a `platform_all` policy, and explicit grants in their migration — there are no default privileges, so a table without them is unreachable by the app.